Bab 13

Implementasi GUI

Memberi tampilan pada aplikasi yang sudah dibangun.

13

DI BAB INI

- Implementasi GUI
- Komponen GUI
- Event handling
- Integrasi GUI dengan class OOP

Pada Bab 12, petugas perpustakaan sudah dapat mendaftarkan buku dan anggota, mencatat peminjaman, mengembalikan buku, serta menyimpan data melalui menu terminal. Sekarang kita menyediakan pekerjaan yang sama melalui jendela, formulir, tombol, dan tabel. Class domain serta layanan tetap menjadi dasar aplikasi; yang kita tambahkan adalah cara pengguna berinteraksi dengannya.

Bab ini melanjutkan dasar Tkinter pada Bab 10 dan rancangan aplikasi pada Bab 11. Hasilnya adalah GUI Sistem Peminjaman Perpustakaan dengan tiga tab: Buku, Anggota, dan Peminjaman. Seluruh kode tambahan ditempatkan dalam satu file `gui.py` agar hubungan antarkomponen mudah ditelusuri sebelum tampilan dipecah menjadi beberapa module.

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- mengubah kebutuhan aplikasi menjadi susunan formulir dan tabel;
- memilih widget dan mengatur hubungan widget dengan data masukan;
- menulis event handler yang membaca input, memanggil layanan, dan memperbarui tampilan;
- menghubungkan GUI dengan package OOP tanpa menggandakan aturan transaksi;
- menangani kegagalan operasi, penyimpanan, dan penutupan jendela;
- memeriksa kesesuaian data di layar dengan keadaan layanan serta file simpanan.

13.1 Implementasi GUI

Menentukan Pekerjaan Pengguna

Implementasi GUI dimulai dengan pekerjaan yang ingin diselesaikan pengguna. Petugas membutuhkan tempat untuk mengisi data, melihat hasil pendaftaran, memilih buku yang tersedia, dan mengetahui apakah peminjaman sudah selesai. Tombol dan tabel dipilih untuk mendukung pekerjaan tersebut.

Tab Buku memperlihatkan jenis dan status ketersediaan buku. Tab Anggota memuat ID dan nama anggota. Tab Peminjaman menampilkan tanggal pinjam, jatuh tempo, tanggal kembali, dan status transaksi. Pilihan buku pada formulir hanya memuat kode yang tersedia. Layanan tetap memeriksa ketersediaan ketika transaksi diminta.

Tabel 13.1. Pemetaan kebutuhan ke bagian antarmuka.

Kebutuhan	Bagian GUI	Operasi layanan
Mendaftarkan buku	Formulir kode, judul, dan jenis pada tab Buku	`tambah_buku()`
Mendaftarkan anggota	Formulir ID dan nama pada tab Anggota	`tambah_anggota()`
Meminjam buku	Pilihan anggota, buku tersedia, tanggal, dan tombol Pinjam	`pinjamkan()`
Mengembalikan buku	Pilihan baris riwayat, tanggal, dan tombol pengembalian	`kembalikan()`
Melihat keadaan	Tabel buku, anggota, dan seluruh riwayat	Method daftar dan `tersedia()`
Menyimpan data	Tombol Simpan dan konfirmasi saat keluar	`simpan()`

Menyiapkan Proyek dan Tkinter

Gunakan proyek Bab 12 yang sudah lengkap dan berhasil dijalankan. Tambahkan `gui.py` sejajar dengan `main.py`; isi lengkap file tersebut terdapat pada Subbab Integrasi GUI dengan Class OOP. Package `perpustakaan` tetap dipakai melalui antarmuka publiknya.

Struktur file yang digunakan

```
proyek_perpustakaan/
|-- main.py
|-- gui.py
|-- perpustakaan/
|   |-- __init__.py
|   |-- kesalahan.py
|   |-- model.py
|   |-- penyimpanan.py
|   `-- layanan.py
`-- data/
    `-- perpustakaan.json
```

Gunakan Python 3.10 atau yang lebih baru dengan dukungan Tkinter. Periksa dukungannya dari terminal dengan perintah berikut; sebuah jendela demonstrasi akan terbuka jika Tkinter tersedia. Tutup jendela itu sebelum menjalankan aplikasi.

Memeriksa Tkinter

```
python -m tkinter
```

Tkinter merupakan antarmuka Python untuk Tcl/Tk, tetapi dapat tidak disertakan

pada distribusi Python tertentu. Jika module tidak ditemukan atau jendela tidak dapat dibuka, lengkapi dukungan Tkinter sesuai petunjuk distribusi Python dan jalankan pada lingkungan desktop. Rujukan resmi tersedia pada dokumentasi Tkinter Python.

SATU APLIKASI UNTUK SATU FILE DATA

Versi terminal dan GUI menggunakan lokasi data yang sama, yaitu folder `data` di samping file program. Jalankan salah satunya dalam satu waktu. Aplikasi belum mengatur perubahan serentak dari beberapa proses; dua jendela yang menyimpan data berbeda dapat saling menimpa hasil simpanan. Gunakan salinan proyek untuk latihan agar data percobaan mudah dimulai ulang.

Menentukan Alur Kerja Jendela

Saat aplikasi dimulai, data dimuat sebelum jendela utama ditampilkan. Jika belum ada file, layanan memulai keadaan kosong. Jika file ada tetapi rusak atau tidak dapat dibaca, GUI menampilkan pesan dan berhenti. Keputusan ini mengikuti perilaku aplikasi terminal pada Bab 12.

Selama aplikasi berjalan, perubahan berada di memori. Tombol Simpan menulis keadaan terbaru ke file. Setelah perubahan yang berhasil, teks status mengingatkan bahwa data belum disimpan. Jika pengguna menutup jendela dalam keadaan tersebut, aplikasi menawarkan pilihan menyimpan, keluar tanpa menyimpan, atau membatalkan penutupan.

13.2 Komponen GUI

Memilih Widget sesuai Tugasnya

Contoh ini memakai `tkinter.ttk` untuk widget bertema. Tampilan detail tombol, tab, dan tabel dapat mengikuti sistem operasi. Kita mengatur susunan serta perilakunya melalui kode, tanpa bergantung pada kesamaan warna atau bentuk di setiap komputer.

`StringVar` menghubungkan nilai teks dengan widget melalui opsi `textvariable`. Method `get()` membaca isinya, sedangkan `set()` memperbaruinya. Nilai tanggal di `StringVar` masih berupa teks; GUI mengubahnya menjadi `date` sebelum memanggil layanan.

Menyusun Formulir dan Tabel

Jendela dibagi menjadi judul, area tab yang dapat membesar, serta baris status dan tombol di bagian bawah. Di dalam setiap tab, formulir berada di atas tabel. `pack()` mengatur susunan antarkelompok, sedangkan `grid()` mengatur kolom formulir dan hubungan tabel dengan scrollbar. Keduanya digunakan pada parent yang berbeda.

Pengaturan `fill="both"` dan `expand=True` memberi ruang tambahan kepada area tabel. Pada parent yang memakai `grid()`, bobot baris dan kolom ditetapkan dengan

Tabel 13.2. Widget dan perannya pada aplikasi perpustakaan.

Komponen	Pemakaian
`Tk`	Satu jendela utama yang menjalankan event loop.
`ttk.Notebook`	Mengelompokkan pekerjaan ke tab Buku, Anggota, dan Peminjaman.
`ttk.Frame`	Menampung formulir, tabel, atau baris tombol.
`ttk.Label`	Memberi nama kolom isian, petunjuk, judul, dan pesan status.
`ttk.Entry`	Menerima teks kode, judul, ID, nama, dan tanggal transaksi.
`ttk.Combobox`	Memilih jenis buku, ID anggota terdaftar, dan kode buku tersedia.
`ttk.Button`	Menjalankan pendaftaran, peminjaman, pengembalian, penyimpanan, atau penutupan.
`ttk.Treeview`	Menampilkan data dalam baris dan kolom serta memilih satu catatan.
`ttk.Scrollbar`	Menggeser tabel secara vertikal dan horizontal.
`messagebox`	Melaporkan penolakan atau meminta keputusan sebelum menutup.

`rowconfigure()` serta `columnconfigure()`. Scrollbar tetap tersedia ketika isi tabel lebih panjang atau lebih lebar daripada area yang terlihat.



Gambar 13.1. Skema tab Peminjaman setelah B001 dipinjam. B001 tidak lagi muncul sebagai pilihan buku tersedia. Gambar menunjukkan susunan informasi; bentuk widget sebenarnya mengikuti tema Tkinter pada komputer pembaca.

Memakai Identitas yang Tetap pada Pilihan

Combobox anggota menyimpan ID anggota, dan combobox buku menyimpan kode buku. Nama anggota dan judul dapat dilihat pada tab masing-masing. Pemakaian ID menghindari masalah ketika dua anggota bernama sama atau dua eksemplar buku berjudul sama. Program tidak perlu memotong teks label untuk menebak identitas object.

Pada `Treeview`, setiap baris diberi `iid` yang sama dengan identitas domainnya. Misalnya, baris peminjaman P001 memakai `iid="P001"`. Hasil `selection()` dapat langsung digunakan sebagai ID ketika memanggil layanan. Posisi baris tidak dijadikan ID karena urutannya dapat berubah.

Combobox dibuat dengan `state="readonly"` agar pengguna memilih dari daftar yang disediakan. Tabel memakai `selectmode="browse"` agar hanya satu catatan dipilih untuk pengembalian. Widget tersebut membantu masukan pengguna; aturan tentang keabsahan ID dan keadaan transaksi tetap diperiksa oleh layanan.

13.3 Event Handling

Menghubungkan Tindakan dengan Method

Event handler pada aplikasi berupa method dari object `AplikasiPerpustakaan`. Tombol menerima rujukan method melalui `command`, sedangkan event keyboard atau perubahan pilihan didaftarkan melalui `bind()`. Pendaftaran dilakukan sekali ketika widget dibuat; Tkinter memanggil handler ketika event yang sesuai terjadi.

Tabel 13.3. Event dan handler pada GUI.

Tindakan pengguna	Handler	Hasil
Klik Daftarkan buku	`tambah_buku()`	Membentuk buku sesuai jenis dan mendaftarkannya.
Klik Daftarkan anggota	`tambah_anggota()`	Membentuk anggota dan mendaftarkannya.
Klik Pinjam	`pinjam()`	Membaca pilihan serta tanggal, lalu meminta transaksi baru.
Klik Kembalikan pilihan	`kembalikan()`	Menggunakan ID baris yang dipilih untuk pengembalian.
Pilih baris pada tabel buku	`pilih_buku(event)`	Mengisi pilihan buku jika kode tersebut tersedia.
Klik Simpan atau tekan Ctrl+S	`simpan( event=None)`	Menyimpan data dan memperbarui teks status.
Klik Tutup atau tombol X	`tutup()`	Memeriksa perubahan yang belum disimpan.

RUJUKAN METHOD, BUKAN HASIL PEMANGGILANNYA

Gunakan `command=self.pinjam`, tanpa tanda kurung. Bentuk `command=self.pinjam()` menjalankan method ketika tombol dibuat dan menyerahkan hasilnya sebagai callback. Handler dari `bind()` menerima object event; parameter `event=None` memungkinkan handler tertentu dipakai oleh tombol maupun event keyboard.

Membaca Input dan Menangani Penolakan

Satu operasi GUI mengikuti urutan: baca input, ubah format bila diperlukan, panggil layanan, lalu perbarui tampilan jika berhasil. Fungsi `baca_tanggal()` menerima teks seperti `2026-10-27` dan menghasilkan object `date`. Format yang salah atau tanggal yang tidak ada, seperti 30 Februari, ditolak dengan pesan yang menjelaskan bentuk input yang benar.

Validasi format termasuk pekerjaan antarmuka. Validasi aturan seperti tanggal kembali yang mendahului tanggal pinjam tetap berada pada model. Dengan pembagian ini, kesalahan input dapat dijelaskan kepada pengguna tanpa memindahkan aturan peminjaman ke handler.

Method bantuan `_jalankan()` menerima sebuah fungsi operasi dan pesan keberhasilan. Ia menangkap `PerpustakaanError` serta `ValueError`, menampilkan kotak pesan jika operasi ditolak, lalu mengembalikan `False`. Jika operasi berhasil, ia menandai adanya perubahan, menyegarkan tampilan, dan mengembalikan `True`. Formulir pendaftaran hanya dikosongkan setelah hasilnya `True`, sehingga input yang ditolak masih dapat dikoreksi.

`_jalankan()` tidak menangkap seluruh `Exception`. Kesalahan pemrograman yang belum dikenali perlu diperbaiki, bukan disembunyikan sebagai masukan pengguna yang salah. Handler juga tidak memanggil `input()` atau membuat perulangan `while True`; setelah selesai, kendali kembali ke event loop.

Menyegarkan Tampilan setelah Keadaan Berubah

Method `segarkan()` membaca daftar terbaru dari layanan, mengisi ulang tabel, lalu memperbarui pilihan anggota dan buku tersedia. Jika sebuah kode buku sudah tidak tersedia, pilihan lama untuk kode tersebut dikosongkan. Pemilihan baris yang masih ada dipertahankan melalui `iid`.

Contohnya, setelah P001 dibuat, tabel riwayat menampilkan P001 sebagai aktif, tabel buku menampilkan B001 sebagai dipinjam, dan combobox tidak lagi menawarkan B001. Setelah P001 dikembalikan, riwayat tetap ada dengan status selesai dan B001 kembali tersedia. Semua perubahan layar berasal dari keadaan layanan yang sama.

Menyimpan dan Menutup dengan Keputusan yang Jelas

Attribute `berubah` menandai adanya operasi yang berhasil sejak penyimpanan terakhir. Ini adalah keadaan antarmuka, berbeda dari property `aktif` pada transaksi. Memilih baris atau mengganti isi formulir yang belum dikirim tidak mengubah data domain dan tidak menandai perubahan yang perlu disimpan.

Ketika `simpan()` berhasil, penanda menjadi `False`. Ketika penyimpanan gagal, penanda tetap seperti sebelumnya dan jendela tetap terbuka. Tombol X dihubungkan ke `tutup()` melalui protokol `WM_DELETE_WINDOW`, sehingga pemeriksaan tersebut berlaku juga ketika pengguna menutup lewat bingkai jendela.

`askyesnocancel()` menghasilkan `True`, `False`, atau `None`. Pemeriksaan `jawaban is None` membedakan Batal dari Tidak. Menyatukan keduanya dengan `if not jawaban` akan kehilangan perbedaan keputusan tersebut.

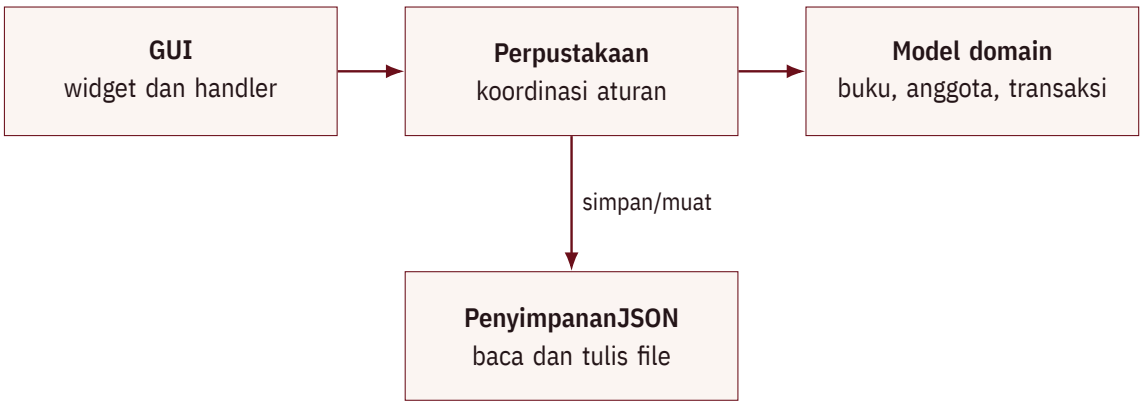
Tabel 13.4. Keputusan saat menutup jendela dengan perubahan belum tersimpan.

Pilihan	Perilaku aplikasi
Ya	Menyimpan terlebih dahulu; jendela ditutup hanya jika penyimpanan berhasil.
Tidak	Menutup tanpa menyimpan perubahan sesi tersebut. File simpanan sebelumnya tetap digunakan pada pembukaan berikutnya.
Batal	Membatalkan penutupan dan kembali ke jendela dengan keadaan yang sama.

13.4 Integrasi GUI dengan Class OOP

Memisahkan Tampilan, Layanan, dan Penyimpanan

Class `AplikasiPerpustakaan` menerima object jendela dan object layanan melalui constructor. Class ini memiliki rujukan ke keduanya serta menyimpan widget dan variabel formulir sebagai attribute. Class domain tidak mewarisi widget dan tidak mengimpor `tkinter`.



Gambar 13.2. GUI memakai layanan yang sama dengan menu terminal. Tanda panah menyatakan arah pemakaian komponen; bukan relasi inheritance.

GUI membuat object `BukuCetak`, `EBook`, atau `Anggota` dari formulir, kemudian mendaftarkannya lewat layanan. Peminjaman baru dibuat oleh layanan melalui `pinjamkan()`, sesuai rancangan composition pada Bab 11. GUI membaca property transaksi untuk menampilkan hasil, tanpa menulis ke dictionary atau attribute internal model.

Pemisahan tersebut memungkinkan pemeriksaan layanan Bab 12 tetap dipakai ketika antarmuka berubah. Pendekatan menyusun dan memeriksa komponen secara bertahap juga dibahas oleh [Wilson, 2024]. Pada bab ini, pemeriksaan GUI ditambahkan untuk menilai hubungan input, callback, dan tampilan yang tidak tercakup oleh pemeriksaan layanan saja.

Kode Lengkap Antarmuka

Buat `gui.py` di folder utama proyek, lalu salin **seluruh listing berikut sebagai isi file tersebut**. Seluruh helper widget, handler, dan fungsi awal aplikasi sudah disertakan. Tidak ada perubahan yang diperlukan pada package `perpustakaan` dari Bab 12.

gui.py

```
from datetime import date
from pathlib import Path
import tkinter as tk
from tkinter import messagebox, ttk

from perpustakaan import (
    Anggota, BukuCetak, EBook, Perpustakaan,
    PerpustakaanError, PenyimpananJSON,
)


def baca_tanggal(teks):
    teks = teks.strip()
    try:
        tanggal = date.fromisoformat(teks)
        if tanggal.isoformat() != teks:
            raise ValueError
    except ValueError as exc:
        raise ValueError("Gunakan tanggal sah berformat YYYY-MM-DD.") from exc
    return tanggal


class AplikasiPerpustakaan:
    def __init__(self, root, perpus):
        self.root = root
        self.perpus = perpus
        self.berubah = False
        root.title("Sistem Peminjaman Perpustakaan")
        root.geometry("1100x760")
        root.minsize(950, 650)

        self.kode = tk.StringVar(root)
        self.judul = tk.StringVar(root)
        self.jenis = tk.StringVar(root, value="cetak")
        self.id_anggota = tk.StringVar(root)
        self.nama = tk.StringVar(root)
        self.anggota_dipilih = tk.StringVar(root)
        self.buku_dipilih = tk.StringVar(root)
        self.tanggal = tk.StringVar(root, value=date.today().isoformat())
        self.status = tk.StringVar(root, value="Siap. Belum ada perubahan.")

```

```
        ttk.Label(root, text="Sistem Peminjaman Perpustakaan",
                  font=("TkDefaultFont", 16, "bold")).pack(pady=(12, 4))
        self.tab = ttk.Notebook(root)
        self.tab.pack(fill="both", expand=True, padx=12, pady=8)
        self._buat_tab_buku()
        self._buat_tab_anggota()
        self._buat_tab_pinjam()

        bawah = ttk.Frame(root, padding=(12, 0, 12, 12))
        bawah.pack(fill="x")
        ttk.Label(bawah, textvariable=self.status).pack(side="left")
        ttk.Button(bawah, text="Tutup", command=self.tutup).pack(side="right")
        ttk.Button(bawah, text="Simpan", command=self.simpan).pack(
            side="right", padx=8)
        root.bind("<Control-s>", self.simpan)
        root.protocol("WM_DELETE_WINDOW", self.tutup)
        self.segarkan()

    def _halaman(self, judul):
        halaman = ttk.Frame(self.tab, padding=12)
        self.tab.add(halaman, text=judul)
        return halaman

    def _isian(self, parent, label, variabel, kolom, lebar=18):
        ttk.Label(parent, text=label).grid(
            row=0, column=kolom, sticky="w", pady=(0, 4))
        isian = ttk.Entry(parent, textvariable=variabel, width=lebar)
        isian.grid(row=1, column=kolom, sticky="ew", padx=(0, 8))
        parent.columnconfigure(kolom, weight=1)
        return isian

    def _tabel(self, parent, kolom):
        wadah = ttk.Frame(parent)
        wadah.pack(fill="both", expand=True, pady=(12, 0))
        wadah.rowconfigure(0, weight=1)
        wadah.columnconfigure(0, weight=1)
        nama = [k[0] for k in kolom]
        tabel = ttk.Treeview(wadah, columns=nama, show="headings",
                             selectmode="browse")
        for nama, judul, lebar in kolom:
            tabel.heading(nama, text=judul)
            tabel.column(nama, width=lebar, minwidth=60, anchor="w")
        vertikal = ttk.Scrollbar(wadah, orient="vertical", command=tabel.yview)
        horizontal = ttk.Scrollbar(wadah, orient="horizontal", command=tabel.
    xview)
        tabel.configure(yscrollcommand=vertikal.set,
```

```
                        xscrollcommand=horizontal.set)
        tabel.grid(row=0, column=0, sticky="nsew")
        vertikal.grid(row=0, column=1, sticky="ns")
        horizontal.grid(row=1, column=0, sticky="ew")
        return tabel

    def _buat_tab_buku(self):
        halaman = self._halaman("Buku")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        self._isian(form, "Kode buku", self.kode, 0, 12)
        self._isian(form, "Judul", self.judul, 1, 36)
        ttk.Label(form, text="Jenis").grid(row=0, column=2, sticky="w")
        ttk.Combobox(form, textvariable=self.jenis, values=("cetak", "ebook"),
                     state="readonly", width=12).grid(row=1, column=2, padx=8)
        ttk.Button(form, text="Daftarkan buku", command=self.tambah_buku).grid(
            row=1, column=3)
        self.tabel_buku = self._tabel(halaman, [
            ("kode", "Kode", 100), ("judul", "Judul", 380),
            ("jenis", "Jenis", 100), ("status", "Status", 110),
        ])
        self.tabel_buku.bind("<<TreeviewSelect>>", self.pilih_buku)

    def _buat_tab_anggota(self):
        halaman = self._halaman("Anggota")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        self._isian(form, "ID anggota", self.id_anggota, 0, 14)
        self._isian(form, "Nama", self.nama, 1, 40)
        ttk.Button(form, text="Daftarkan anggota", command=self.tambah_anggota).
    grid(
            row=1, column=2)
        self.tabel_anggota = self._tabel(halaman, [
            ("id", "ID anggota", 130), ("nama", "Nama", 500),
        ])

    def _buat_tab_pinjam(self):
        halaman = self._halaman("Peminjaman")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        for kolom, label in enumerate(("ID anggota", "Kode buku tersedia")):
            ttk.Label(form, text=label).grid(row=0, column=kolom, sticky="w")
        self.pilihan_anggota = ttk.Combobox(
            form, textvariable=self.anggota_dipilih, state="readonly", width=16)

        self.pilihan_anggota.grid(row=1, column=0, padx=(0, 8))
```

```
131         self.pilihan_buku = ttk.Combobox(
132             form, textvariable=self.buku_dipilih, state="readonly", width=16)
133         self.pilihan_buku.grid(row=1, column=1, padx=(0, 8))
134         self._isian(form, "Tanggal transaksi (YYYY-MM-DD)", self.tanggal, 2, 20)
135         ttk.Button(form, text="Pinjam", command=self.pinjam).grid(
136             row=1, column=3, padx=4)
137         ttk.Button(form, text="Kembalikan pilihan", command=self.kembalikan).
    grid(
138             row=1, column=4)
139         ttk.Label(halaman, text="Untuk pengembalian, pilih satu baris riwayat "
140                   "dan isi tanggal transaksi.").pack(anchor="w", pady=(12, 0))
141         self.tabel_pinjam = self._tabel(halaman, [
142             ("id", "ID pinjam", 80), ("anggota", "Anggota", 90),
143             ("buku", "Buku", 80), ("pinjam", "Tanggal pinjam", 115),
144             ("jatuh", "Jatuh tempo", 115), ("kembali", "Tanggal kembali", 120),
145             ("status", "Status", 85),
146         ])
147
148     def _isi_tabel(self, tabel, baris):
149         dipilih = tabel.selection()
150         tabel.delete(*tabel.get_children())
151         for identitas, nilai in baris:
152             tabel.insert("", "end", iid=identitas, values=nilai)
153         if dipilih and tabel.exists(dipilih[0]):
154             tabel.selection_set(dipilih[0])
155
156     def segarkan(self):
157         buku = self.perpus.daftar_buku()
158         anggota = self.perpus.daftar_anggota()
159         self._isi_tabel(self.tabel_buku, [
160             (b.kode, (b.kode, b.judul, b.JENIS,
161                       "tersedia" if self.perpus.tersedia(b.kode) else "dipinjam
    "))
162             for b in buku
163         ])
164         self._isi_tabel(self.tabel_anggota, [
165             (a.id_anggota, (a.id_anggota, a.nama)) for a in anggota
166         ])
167         self._isi_tabel(self.tabel_pinjam, [
168             (p.id_pinjam, (p.id_pinjam, p.anggota.id_anggota, p.buku.kode,
169                            p.tanggal_pinjam.isoformat(),
170                            p.tanggal_jatuh_tempo.isoformat(),
171                            p.tanggal_kembali.isoformat() if p.tanggal_kembali
    else "-",
172                            "aktif" if p.aktif else "selesai"))
```

```
            for p in self.perpus.daftar_peminjaman()
        ])
        id_anggota = [a.id_anggota for a in anggota]
        tersedia = [b.kode for b in buku if self.perpus.tersedia(b.kode)]
        self.pilihan_anggota["values"] = id_anggota
        self.pilihan_buku["values"] = tersedia
        if self.anggota_dipilih.get() not in id_anggota:
            self.anggota_dipilih.set("")
        if self.buku_dipilih.get() not in tersedia:
            self.buku_dipilih.set("")

    def _jalankan(self, aksi, pesan):
        try:
            aksi()
        except (PerpustakaanError, ValueError) as exc:
            messagebox.showerror("Operasi ditolak", str(exc), parent=self.root)
            return False
        self.berubah = True
        self.segarkan()
        self.status.set(f"{pesan} Perubahan belum disimpan.")
        return True

    def tambah_buku(self):
        def aksi():
            kelas = {"cetak": BukuCetak, "ebook": EBook}.get(self.jenis.get())
            if kelas is None:
                raise ValueError("Pilih jenis cetak atau ebook.")
            self.perpus.tambah_buku(kelas(self.kode.get(), self.judul.get()))
        if self._jalankan(aksi, "Buku terdaftar."):
            self.kode.set("")
            self.judul.set("")

    def tambah_anggota(self):
        aksi = lambda: self.perpus.tambah_anggota(
            Anggota(self.id_anggota.get(), self.nama.get()))
        if self._jalankan(aksi, "Anggota terdaftar."):
            self.id_anggota.set("")
            self.nama.set("")

    def pinjam(self):
        def aksi():
            if not self.anggota_dipilih.get() or not self.buku_dipilih.get():
                raise ValueError("Pilih anggota dan buku yang tersedia.")
            self.perpus.pinjamkan(self.anggota_dipilih.get(),
                                  self.buku_dipilih.get(),
                                  baca_tanggal(self.tanggal.get()))
```

```
219             self._jalankan(aksi, "Peminjaman tercatat.")
220
221         def kembalikan(self):
222             def aksi():
223                 pilihan = self.tabel_pinjam.selection()
224                 if not pilihan:
225                     raise ValueError("Pilih satu baris peminjaman.")
226                 self.perpus.kembalikan(pilihan[0], baca_tanggal(self.tanggal.get()))
227             self._jalankan(aksi, "Pengembalian tercatat.")
228
229         def pilih_buku(self, event=None):
230             pilihan = self.tabel_buku.selection()
231             if pilihan:
232                 kode = pilihan[0]
233                 self.buku_dipilih.set(kode if self.perpus.tersedia(kode) else "")
234
235         def simpan(self, event=None):
236             try:
237                 self.perpus.simpan()
238             except PerpustakaanError as exc:
239                 messagebox.showerror("Gagal menyimpan", str(exc), parent=self.root)
240                 return False
241             self.berubah = False
242             self.status.set("Data tersimpan.")
243             return True
244
245         def tutup(self):
246             if self.berubah:
247                 jawaban = messagebox.askyesnocancel(
248                     "Simpan perubahan?", "Simpan perubahan sebelum keluar?",
249                     parent=self.root)
250                 if jawaban is None:
251                     return
252                 if jawaban and not self.simpan():
253                     return
254             self.root.destroy()
255
256
257 def main():
258     root = tk.Tk()
259     root.withdraw()
260     path = Path(__file__).resolve().parent / "data" / "perpustakaan.json"
261     perpus = Perpustakaan(PenyimpananJSON(path))
262     try:
263         perpus.muat()
```

```
264     except PerpustakaanError as exc:
265         messagebox.showerror("Data tidak dimuat", str(exc), parent=root)
266         root.destroy()
267         return 1
268     AplikasiPerpustakaan(root, perpus)
269     root.deiconify()
270     root.mainloop()
271     return 0
272
273
274 if __name__ == "__main__":
275     raise SystemExit(main())
```

Method yang diawali garis bawah dipakai untuk menyusun bagian tampilan atau menjalankan pekerjaan bantuan. Method publik seperti `pinjam()`, `kembalikan()`, dan `simpan()` menjadi callback. Fungsi lokal `aksi()` serta `lambda` menunda pemanggilan layanan sampai fungsi itu dijalankan oleh `_jalankan()`.

Fungsi `main()` membuat satu object `Tk`, menyembunyikannya sementara selama pemuatan, lalu menampilkan jendela setelah layanan siap. Jika pemuatan gagal, kotak pesan ditampilkan dengan parent yang jelas dan jendela dihancurkan. Jika berhasil, `mainloop()` menangani interaksi sampai pengguna menutup aplikasi.

Menjalankan dan Menelusuri Hasilnya

Jalankan dari folder proyek dengan perintah berikut. Hasil program berupa jendela, tabel, teks status, dan kotak pesan; tidak ada rangkaian `print()` seperti pada demo terminal.

Perintah menjalankan aplikasi

```
python gui.py
```

Untuk memperoleh hasil yang dapat dibandingkan, gunakan salinan proyek dengan folder data baru, lalu ikuti langkah berikut.

Mencoba Satu Siklus Peminjaman

1. Pada tab Buku, daftarkan B001 berjudul Dasar OOP dengan jenis cetak dan E001 berjudul Dasar Python dengan jenis ebook.
2. Pada tab Anggota, daftarkan M001 dengan nama Ani.
3. Pada tab Peminjaman, pilih M001 dan B001, isi tanggal `2026-10-27`, lalu klik Pinjam.
4. Periksa P001: tanggal jatuh tempo `2026-11-10`, tanggal kembali berupa tanda `-`, dan status aktif. B001 ditampilkan sebagai dipinjam dan tidak lagi muncul pada pilihan buku tersedia.

5. Pilih baris P001, ubah tanggal transaksi menjadi `2026-11-02`, lalu klik Kembalikan pilihan. P001 menjadi selesai dan B001 tersedia kembali.
6. Pilih M001 dan E001, gunakan tanggal `2026-11-03`, lalu klik Pinjam. Transaksi P002 jatuh tempo pada `2026-11-10`, sesuai masa pinjam e-book tujuh hari.
7. Klik Simpan, tutup aplikasi, lalu buka kembali. Kedua buku, anggota, dan dua catatan riwayat harus tetap terlihat.

Tabel 13.5. Keadaan layar yang diharapkan pada percobaan.

Tahap	Buku	Riwayat
Pendaftaran selesai	B001 dan E001 tersedia	Belum ada transaksi
B001 dipinjam	B001 dipinjam; E001 tersedia	P001 aktif, jatuh tempo 10 November
B001 dikembalikan	B001 dan E001 tersedia	P001 selesai, kembali 2 November
E001 dipinjam	B001 tersedia; E001 dipinjam	P001 selesai dan P002 aktif
Simpan dan buka ulang	Sama dengan keadaan terakhir tersimpan	ID, tanggal, dan status tetap sama

Tanggal awal pada formulir diisi dengan tanggal komputer saat aplikasi dibuka. Pembaca harus menggantinya dengan tanggal pada langkah percobaan jika ingin mencocokkan hasil tersebut. ID P001 dan P002 berlaku untuk data baru; proyek yang sudah memiliki riwayat akan melanjutkan nomor yang tersimpan.

Memeriksa Penolakan dan Penutupan

Periksa pula masukan dan keputusan yang gagal. Keberhasilan GUI mencakup kemampuan menjelaskan penolakan sambil mempertahankan data yang benar. Gunakan tabel berikut sebagai daftar skenario yang dapat diulang pada salinan proyek.

Daftar pilihan buku tersedia merupakan bantuan tampilan, bukan pengganti pemeriksaan transaksi. Larangan peminjaman ganda sudah diperiksa pada layanan Bab 12 dan tetap berlaku ketika method dipanggil dari antarmuka lain. Pengujian GUI melengkapi pemeriksaan tersebut dengan menilai pilihan widget, perubahan tabel, pesan, dan keputusan penutupan.

JIKA JENDELA TERASA TIDAK MENANGGAPI

Handler berjalan pada event loop yang sama dengan tampilan. Pekerjaan yang lama dapat menunda respons jendela. Contoh ini membaca dan menulis file lokal berukuran kecil. Jika kelak

Tabel 13.6. Pemeriksaan GUI di luar alur normal.

Percobaan	Hasil yang diharapkan
Daftarkan kode buku yang sama	Pesan penolakan; buku lama tetap ada dan isian masih dapat dikoreksi.
Pinjam tanpa memilih anggota atau buku	Pesan meminta pilihan; tidak ada transaksi baru.
Isi tanggal `2026-02-30`	Pesan format/tanggal tidak sah; riwayat tidak berubah.
Kembalikan tanpa memilih baris	Pesan meminta satu baris; tidak ada catatan yang berubah.
Kembalikan sebelum tanggal pinjam	Layanan menolak; transaksi tetap aktif.
Kembalikan catatan yang sudah selesai	Layanan menolak; tanggal kembali sebelumnya tetap ada.
Tutup setelah perubahan, lalu pilih Batal	Jendela tetap terbuka dengan data dan penanda perubahan yang sama.
Pilih Ya, tetapi penyimpanan gagal	Pesan kegagalan; jendela tetap terbuka dan perubahan belum dianggap tersimpan.
Buka salinan file JSON yang rusak	Pesan pemuatan gagal; jendela utama tidak melanjutkan dengan data kosong.

ada unduhan atau pengolahan besar, pelajari pembagian pekerjaan dan penjadwalan pembaruan GUI; jangan menambahkan `sleep()` atau perulangan panjang ke handler.

13.5 Rangkuman

- GUI menerjemahkan pekerjaan pengguna menjadi formulir, pilihan, tombol, tabel, dan umpan balik yang jelas.
- `StringVar` menghubungkan teks dengan widget. Konversi format dilakukan sebelum nilai diteruskan ke layanan.
- Event handler didaftarkan sebagai rujukan method. `command` dan `bind()` mempunyai cara pemberian argumen yang berbeda.
- ID domain dipakai sebagai nilai pilihan dan identitas baris tabel, sehingga operasi tidak bergantung pada urutan baris atau keunikan nama.
- Tampilan disegarkan dari layanan setelah operasi berhasil; operasi yang ditolak tidak dianggap sebagai perubahan data baru.
- Model dan layanan menjaga aturan OOP, sedangkan GUI membaca input, memanggil operasi, dan menyajikan hasilnya.
- Penyimpanan yang gagal serta pembatalan penutupan tidak boleh dianggap sebagai keberhasilan menyimpan data.

13.6 Kuis Teoritis

Pilihan Ganda

1. Pendaftaran callback tombol yang benar adalah ...

   a. `command=self.pinjam()`
   b. `command=self.pinjam`
   c. `command="self.pinjam()"`
   d. `command=None` untuk menjalankan peminjaman

2. Keuntungan memakai ID peminjaman sebagai `iid` tabel adalah ...

   a. identitas catatan tetap dapat ditemukan walaupun urutan baris berubah
   b. semua tanggal otomatis menjadi sah
   c. layanan tidak lagi diperlukan
   d. catatan selesai otomatis dihapus

3. Pemeriksaan buku masih tersedia ketika transaksi dibuat menjadi tanggung jawab ...

a. warna tombol
b. teks judul jendela
c. layanan `Perpustakaan`
d. scrollbar tabel

4. Jika pengguna memilih Batal pada dialog penutupan, aplikasi harus ...

a. menutup tanpa menyimpan
b. menghapus perubahan di memori
c. menganggap penyimpanan berhasil
d. tetap membuka jendela dengan keadaan yang sama

5. Pernyataan yang tepat setelah pengembalian berhasil adalah ...

a. seluruh riwayat peminjaman buku dihapus
b. layanan berubah, kemudian GUI membaca keadaan terbaru untuk ditampilkan
c. GUI cukup mengubah tulisan status tanpa memanggil layanan
d. tanggal kembali pada model tetap kosong

KUNCI PILIHAN GANDA

1 b; 2 a; 3 c; 4 d; 5 b.

Uraian

1. Jelaskan perbedaan pemeriksaan format tanggal di GUI dan pemeriksaan tanggal kembali pada model.
2. Mengapa pilihan buku perlu diperbarui setelah peminjaman maupun pengembalian?
3. Telusuri apa yang terjadi jika pengguna memilih Ya saat menutup, tetapi penulisan file gagal. Sebutkan keadaan jendela dan penanda perubahan.

13.7 Praktik Mandiri

Praktik 1: Menyaring Riwayat Aktif

Tambahkan `ttk.Checkbutton` untuk menampilkan hanya peminjaman aktif. Gunakan parameter `aktif_saja` pada layanan. Hasil wajib: catatan selesai tersembunyi ketika filter aktif dan muncul kembali ketika filter dimatikan; tidak ada catatan domain yang dihapus.

Praktik 2: Mempermudah Pencarian Buku

Tambahkan isian pencarian judul pada tab Buku. Cocokkan teks tanpa membedakan huruf besar dan kecil, lalu tampilkan hasilnya pada tabel. Pertahankan kode buku sebagai `iid`. Hasil wajib: pencarian tidak mengubah koleksi layanan dan pemilihan baris hasil tetap mengarah ke buku yang benar.

Praktik 3: Memeriksa Keputusan Penutupan

Buat catatan pemeriksaan untuk kondisi tanpa perubahan, perubahan belum disimpan, Batal, Tidak, Ya berhasil, dan Ya dengan penyimpanan gagal. Gunakan salinan proyek atau penyimpanan percobaan. Hasil wajib: hanya penyimpanan berhasil yang menghapus penanda perubahan, dan kegagalan menyimpan tidak menutup jendela.

Nilai hasil praktik dari ketepatan perilaku, pemakaian method publik layanan, keterbacaan susunan widget, dan bukti pemeriksaan. Setelah alur tersebut dipahami, pembaca dapat memecah class GUI menjadi module tampilan per tab tanpa mengubah kontrak layanan.

REFERENSI TEKNIS LANJUTAN

Pelajari dokumentasi resmi Tkinter untuk hubungan widget, event loop, dan handler. Rincian `Notebook`, `Combobox`, `Treeview`, serta scrollbar dapat dibaca pada dokumentasi resmi `tkinter.ttk`. Gunakan dokumentasi yang sesuai dengan versi Python yang dipakai.

Daftar Pustaka

Sumber yang dirujuk dalam bab ini.

[Wilson, 2024] Wilson, G. (2024). *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.